

COMBINATIONAL LOGIC DESIGN

Module.2

CDA 3103C

Spring 2026

Mesut Ozdag, Ph.D.

Notation for incompletely specified functions

- Don't cares and canonical forms
 - So far, only represented on-set
 - Also represent don't-care-set
 - Need two of the three sets (on-set, off-set, dc-set)
- Canonical representations of the BCD increment by 1 function:
 - $Z = m_0 + m_2 + m_4 + m_6 + m_8 + d_{10} + d_{11} + d_{12} + d_{13} + d_{14} + d_{15}$
 - $Z = \Sigma [m(0,2,4,6,8) + d(10,11,12,13,14,15)]$
 - $Z = M_1 \cdot M_3 \cdot M_5 \cdot M_7 \cdot M_9 \cdot D_{10} \cdot D_{11} \cdot D_{12} \cdot D_{13} \cdot D_{14} \cdot D_{15}$
 - $Z = \Pi [M(1,3,5,7,9) \cdot D(10,11,12,13,14,15)]$

Simplification of two-level combinational logic

- Recall that canonical forms guarantee us 2 levels of logic. However, canonical forms do not guarantee us the most minimal version of the function.

- canonical form $\neq$ minimal form

$$F(A, B, C) = (A + B + C) (A + B' + C) (A' + B + C)$$

$$\text{Idempotency} = (A + B + C) (A + B' + C) (A + B + C) (A' + B + C)$$

$$\text{Uniting} = (A + C) (B + C)$$

Uniting:

$$X \cdot Y + X \cdot Y' = X$$

$$(X + Y) \cdot (X + Y') = X$$

Simplification of two-level combinational logic

- The goal is to find a minimal sum of products or product of sums realization
 - Exploit don't care information in the process
- Algebraic simplification
 - Not an algorithmic/systematic procedure
 - How do you know when the minimum realization has been found?
- Computer-aided design tools
 - Precise solutions require very long computation times, especially for functions with many inputs (> 10)
 - Heuristic methods employ "educated guesses" to reduce amount of computation and yield good if not best solutions
- Hand methods still relevant
 - To understand automatic tools and their strengths and weaknesses
 - Ability to check results (on small examples)

The uniting theorem

- Key tool to simplification: $A(B' + B) = A$
- Essence of simplification of two-level logic
 - Find two element subsets of the ON-set where only one variable changes its value – this single varying variable can be eliminated and a single product term used to represent both elements

$$F = A'B' + AB' = (A' + A)B' = B'$$

A	B	F
0	0	1
0	1	0
1	0	1
1	1	0

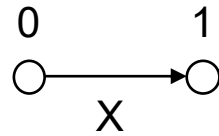
B has the same value in both on-set rows
- B remains

A has a different value in the two rows
- A is eliminated

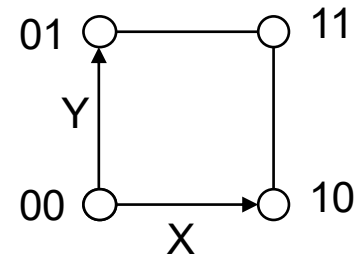
Boolean cubes

- Visual technique for identifying when the uniting theorem can be applied
- n input variables = n -dimensional "cube"

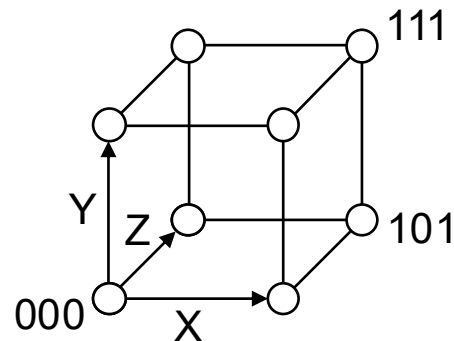
1-cube



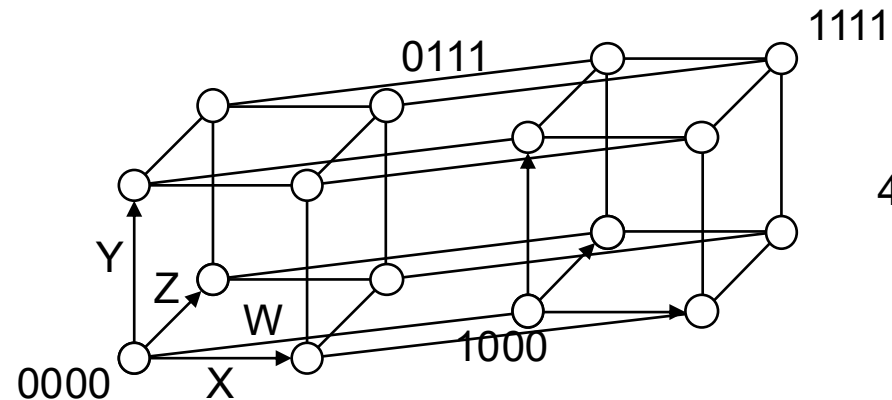
2-cube



3-cube



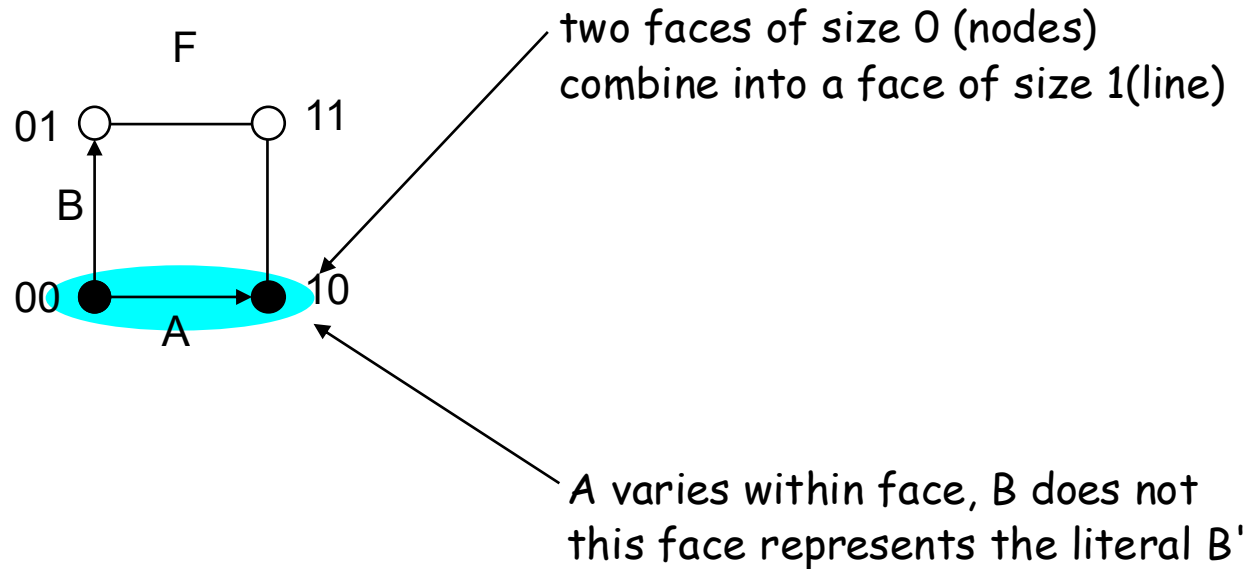
4-cube



Mapping truth tables onto Boolean cubes

- Uniting theorem combines two "faces" of a cube into a larger "face"
- Example:

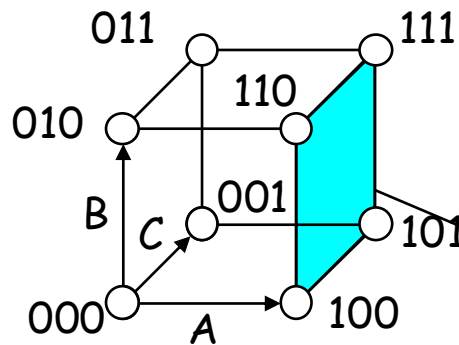
A	B	F
0	0	1
0	1	0
1	0	1
1	1	0



ON-set = solid nodes
OFF-set = empty nodes
DC-set = x'd nodes

Higher dimensional cubes

- Sub-cubes of higher dimension than 2



$$F(A,B,C) = \sum m(4,5,6,7)$$

on-set forms a square
i.e., a cube of dimension 2

represents an expression in one variable
i.e., 3 dimensions - 2 dimensions

A is asserted (true) and unchanged
B and C vary

This subcube represents the
literal A

In a 3-cube (three variables):

- A 0-cube (a single node) yields a term in 3 literals
- A 1-cube (a line of two nodes) yields a term in 2 literals
- A 2-cube (a plane of four nodes) yields a term in 1 literal
- A 3-cube (a cube of eight nodes) yields a constant term "1"

Karnaugh maps

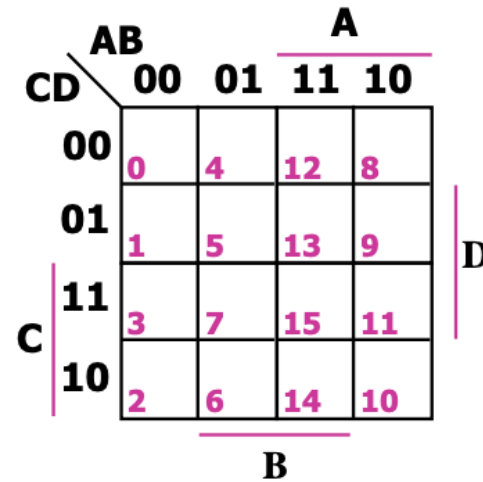
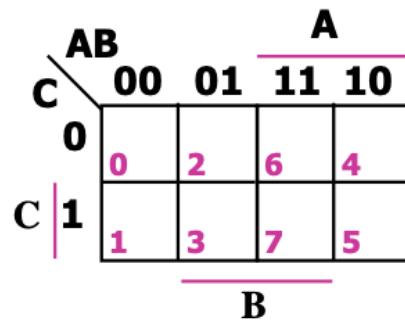
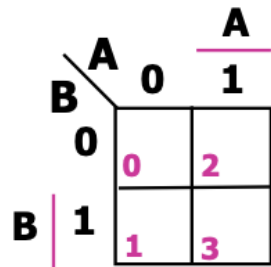
- Flat map of Boolean cube
 - Wrap-around at edges
 - Hard to draw and visualize for more than 4 dimensions
 - Virtually impossible for more than 6 dimensions
- Alternative to truth-tables to help visualize adjacencies
 - Guide to applying the uniting theorem
 - On-set elements with only one variable changing value are adjacent unlike the situation in a linear truth-table

B \ A	0	1
0	1 0	1 2
1	0 1	0 3

A	B	F
0	0	1
0	1	0
1	0	1
1	1	0

K-map cell numbering

- ◆ Gray-code: Only one bit changes between cells
 - Example: 00 → 01 → 11 → 10
- ◆ Layout for 2 – 4 dimension K-maps:

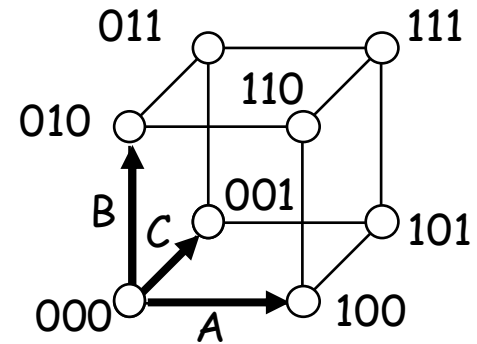
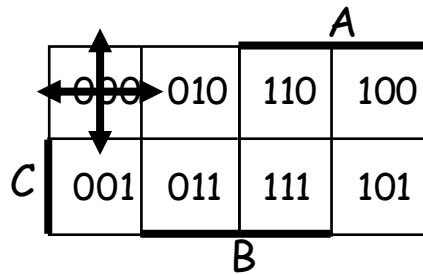


Gray Code		Natural Binary	
0000	0	0000	0
0001	1	0001	1
0011	2	0010	2
0010	3	0011	3

$$13 = 1101 = ABC'D$$

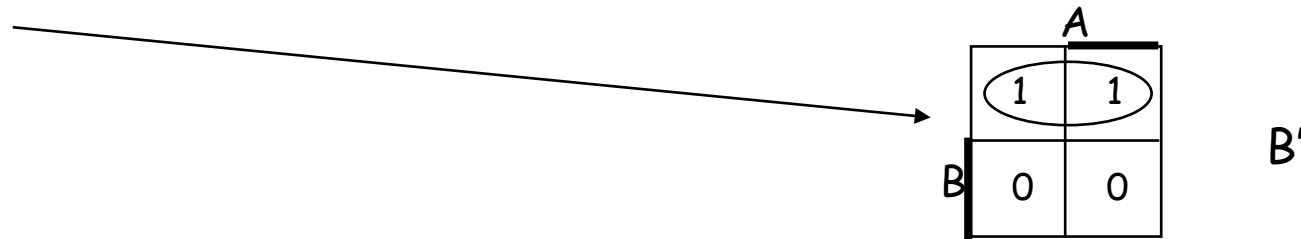
Adjacencies in Karnaugh maps

- Wrap from first to last column
- Wrap top row to bottom row

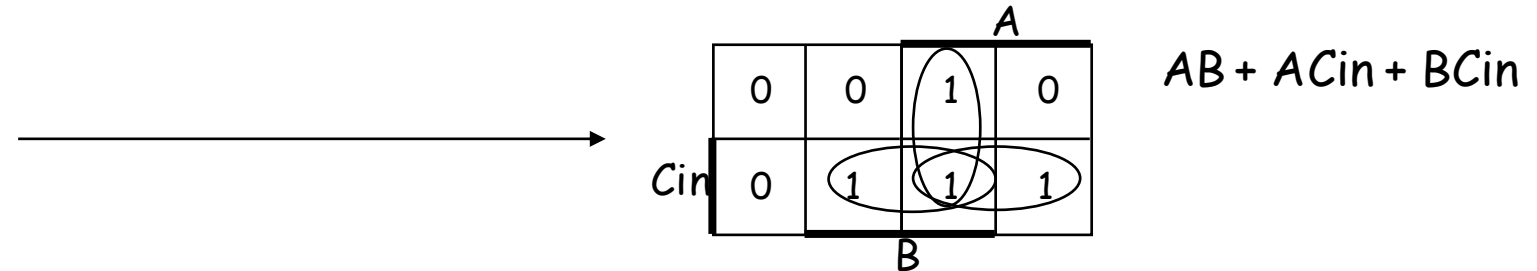


Karnaugh map examples

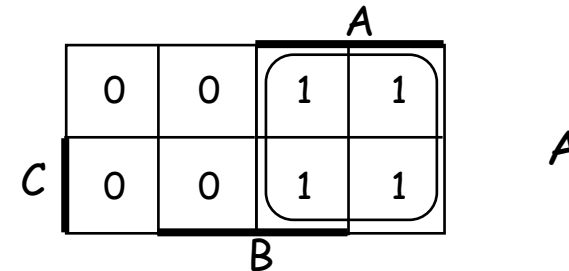
- $F =$



- $C_{out} =$



- $G(A, B, C) =$



K-map minimization (con't)

- ◆ Obtain the complement by covering 0s with subcubes

AB		A			
		00	01	11	10
C	0	1 0	0 2	0 6	1 4
	1	0 1	0 3	1 7	1 5
		B			

$$F(A,B,C) = \sum m(0,4,5,7) \\ = B'C' + AC$$

$$F'(A,B,C) = \sum m(1,2,3,6) \\ = A'C + BC'$$

AB		A			
		00	01	11	10
C	0	1	0	1	1
	1	0	0	1	1

$$F(A,B,C) = ???$$

$$F'(A,B,C) = ???$$

$$F = \sum m(0,4,5,6,7) = A + B'C'$$

$$F' = \sum m(1,2,3) = A'C + A'B = A'(B+C)$$



MESUT OZDAG, PH.D.
MESUT.OZDAG@UCF.EDU

